SECURITY CONFIGURATION GUIDE

MAXIMUM SECURITY

# Ai-Arena VPS Hardening Guide

A comprehensive, step-by-step security hardening reference for Ai-Arena platform deployments. Covers SSH hardening, firewall rules, encryption architecture, intrusion detection, and incident response procedures to protect your infrastructure against modern threats.

Platform: Ai-Arena
Classification: Internal Security Reference
Version 1.0 — 2025

# Table of Contents

# 1. Introduction

Virtual Private Servers (VPS) form the backbone of the Ai-Arena platform, serving as the primary infrastructure for hosting the game management server, player authentication services, real-time matchmaking, and encrypted communication channels between players and the game master. Because a VPS has a publicly routable IP address, it is inherently exposed to the global internet. Unlike internal corporate servers shielded by multiple layers of network segmentation, a VPS sits directly on the public network perimeter, making it one of the most targeted assets in any deployment architecture. Every exposed service, every open port, and every running daemon represents a potential attack surface that adversaries can probe, exploit, and compromise.

The threat model for the Ai-Arena platform is multifaceted. At the highest level, the primary adversaries include automated botnets that continuously scan the entire IPv4 address space for vulnerable SSH configurations, outdated web server software, and misconfigured databases. These botnets deploy credential-stuffing attacks using leaked password dictionaries, exploit known vulnerabilities in common software stacks, and attempt distributed denial-of-service (DDoS) attacks to disrupt service availability. Beyond automated threats, the platform must also account for targeted attacks from competitors or malicious actors who may attempt to manipulate game outcomes, intercept encrypted WebSocket traffic, or compromise the game master server to inject fraudulent game state data. The financial and reputational damage from such compromises can be severe, including loss of player trust, regulatory penalties, and direct financial losses from manipulated game economies.

This VPS Hardening Guide provides a comprehensive, defense-in-depth approach to securing the Ai-Arena platform server. The guide follows a layered security model where each layer independently reduces risk: SSH hardening eliminates the most common initial attack vector, firewall rules restrict network access to only essential services, encryption ensures data confidentiality even if network traffic is intercepted, and monitoring provides early detection of compromise. Each section includes specific configuration commands, recommended settings, and the rationale behind every security decision. By implementing these measures systematically, the Ai-Arena platform achieves a security posture that significantly exceeds industry baselines and provides robust protection against both opportunistic and targeted attacks. The guide is designed for system administrators with intermediate Linux experience and assumes an Ubuntu 22.04 LTS or Debian 12 base installation.

# 2. SSH Hardening

Secure Shell (SSH) is the primary remote administration protocol for any Linux VPS, and it is consistently the most attacked service on public-facing servers. Automated scanners attempt SSH connections within hours of a new server going online, using dictionaries of millions of common passwords and leveraging compromised SSH keys from public data breaches. A poorly configured SSH daemon is the single fastest path to full server compromise. The Ai-Arena platform demands rigorous SSH hardening to eliminate this critical attack vector entirely.

## 2.1 Key-Based Authentication

Password-based authentication must be completely disabled on the Ai-Arena server. Instead, all access must use Ed25519 or RSA 4096-bit SSH key pairs. Ed25519 keys are preferred due to their smaller size, faster authentication, and resistance to side-channel attacks. Each administrator should generate a unique key pair on their local machine and install only the public key on the server. The authorized_keys file must be strictly permissioned at 600, and the ~/.ssh directory at 700. Never share private keys between administrators or store them on shared storage.

```
# Generate Ed25519 key pair (client-side)
ssh-keygen -t ed25519 -C "admin@ai-arena" -f ~/.ssh/ai_arena_ed25519

# Copy public key to server
ssh-copy-id -i ~/.ssh/ai_arena_ed25519.pub user@your-vps-ip

# Set correct permissions on server
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys
```

## 2.2 Disable Root Login

Direct root login via SSH must be disabled. All administrative tasks should be performed by logging in as a standard user and escalating privileges with sudo. This provides an audit trail of who ran privileged commands and when, and it forces attackers to compromise both a user account and the sudo mechanism to gain root access. Create a dedicated administrative user with a strong username (not "admin" or "user") before disabling root login to avoid locking yourself out of the system.

```
# /etc/ssh/sshd_config
PermitRootLogin no
AllowUsers arena-admin
AuthenticationMethods publickey
```

## 2.3 Change SSH Port

Changing the default SSH port from 22 to a non-standard high port (above 1024 and below 65535) reduces the volume of automated brute-force attempts by approximately 98%. While this is defense through obscurity and not a substitute for strong authentication, it dramatically reduces log noise and CPU consumption from botnet scanning. Choose a port that is easy to remember but not commonly used. The port change must be reflected in firewall rules and any automated deployment scripts that connect via SSH.

```
# /etc/ssh/sshd_config
Port 48292

# Update UFW rule
sudo ufw allow 48292/tcp comment "SSH"
sudo systemctl restart sshd
```

## 2.4 Fail2ban for SSH

Fail2ban monitors SSH authentication logs and temporarily bans IP addresses that exceed a configured failure threshold. For the Ai-Arena platform, configure Fail2ban with aggressive settings: a maximum of 3 failed attempts within 10 minutes triggers a 1-hour ban, and repeated offenders are banned for progressively longer durations. The detection filter must be tuned to match the specific log format of your SSH daemon and account for any port changes made above. Fail2ban communicates with the firewall (UFW or iptables) to implement bans at the network level, preventing banned IPs from reaching any service on the server.

### 2.5 Two-Factor Authentication (Optional)

For maximum security, install Google Authenticator PAM module to add time-based one-time password (TOTP) authentication on top of SSH key-based login. This ensures that even if a private key is stolen, the attacker cannot authenticate without the dynamically generated code from the administrator's mobile device. Configure the PAM module in /etc/pam.d/sshd and enable ChallengeResponseAuthentication in sshd_config. Each administrator should run "google-authenticator" on the server to generate their unique secret key and scan the QR code into their authenticator app. Emergency scratch codes should be stored securely offline.

## 3. Firewall Configuration (UFW)

Uncomplicated Firewall (UFW) provides a user-friendly interface for managing iptables/nftables rules and is the recommended firewall solution for the Ai-Arena platform. The fundamental principle of the firewall configuration is default-deny: all incoming traffic is blocked unless explicitly permitted, while all outgoing traffic is allowed (the server needs to initiate connections to external APIs, package repositories, and time servers). This approach ensures that even if a new vulnerability is discovered in a service running on the VPS, it cannot be exploited from the network unless a firewall rule explicitly permits that traffic.

### 3.1 Default Deny Policy

Begin by resetting all existing firewall rules to prevent conflicts from previous configurations, then set the default policies. Incoming traffic defaults to deny, outgoing to allow, and routed traffic (if applicable) to deny. This creates a clean slate where only explicitly allowed traffic can reach the server services. Never run a production server with default-allow incoming policy, even temporarily during configuration. Always apply rules before enabling the firewall to avoid locking yourself out.

```
sudo ufw reset
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw logging low
```

### 3.2 Port List for Ai-Arena

The Ai-Arena platform requires a minimal set of open ports. Every additional open port increases the attack surface and must be justified with a documented business need. The following table lists all required ports and the services they support. Restrict each port to the specific protocol (TCP or UDP) required by the service. Do not open unnecessary ranges or use "allow all" rules.

| Port | Protocol | Service | Purpose |
|---|---|---|---|
| 48292 | TCP | SSH | Remote administration (changed from 22) |
| 80 | TCP | HTTP | Nginx redirect to HTTPS |
| 443 | TCP | HTTPS | Nginx reverse proxy + WebSocket (TLS) |
| 3000 | TCP | Node.js | Ai-Arena game server (localhost only) |
| 5432 | TCP | PostgreSQL | Database (localhost only) |
| 6379 | TCP | Redis | Session cache (localhost only) |

> Important: Ports 3000, 5432, and 6379 must NEVER be exposed to the public internet. They are bound to 127.0.0.1 and accessible only through the Nginx reverse proxy on port 443.

## 3.3 UFW Rules Implementation

```
# Allow SSH on custom port
sudo ufw allow 48292/tcp comment "SSH custom port"

# Allow HTTP/HTTPS for Nginx
sudo ufw allow 80/tcp comment "HTTP redirect"
sudo ufw allow 443/tcp comment "HTTPS + WebSocket"

# Allow loopback
sudo ufw allow in on lo

# Enable firewall
sudo ufw enable

# Verify status
sudo ufw status verbose
```

## 3.4 Logging Dropped Packets

Enable UFW logging at the "low" level to record dropped packets without overwhelming the system log. Low-level logging captures all blocked packets, which is essential for intrusion detection and forensic analysis after an incident. Logs are written to /var/log/ufw.log and can be integrated with log monitoring tools like logwatch or forwarded to a centralized logging server. Periodically review these logs to identify scanning patterns, repeated connection attempts from specific IPs, and attempts to access services that should not be publicly reachable.

```
sudo ufw logging low

# Rate limit SSH to prevent brute-force at firewall level
sudo ufw limit 48292/tcp comment "Rate limit SSH"
```

# 4. Fail2ban Configuration

Fail2ban is a critical intrusion prevention framework that monitors log files for suspicious activity and automatically updates firewall rules to block offending IP addresses. For the Ai-Arena platform, Fail2ban provides the second line of defense after SSH key-based authentication, protecting against distributed brute-force attacks that might eventually guess correct credentials, as well as protecting the Nginx reverse proxy against HTTP-level abuse such as credential stuffing, path traversal attempts, and WebSocket endpoint probing. A properly configured Fail2ban deployment dramatically reduces the noise and risk from automated attack tools.

## 4.1 SSH Jail Configuration

The SSH jail monitors /var/log/auth.log for failed SSH authentication attempts. Configure aggressive ban settings: a maximum of 3 failures within a 10-minute findtime triggers an initial 1-hour bantime. Enable the "recidive" feature to impose exponential penalties on repeat offenders, and use the "ignoreip" directive to whitelist trusted IP addresses such as your office network, CI/CD pipeline servers, and monitoring systems. The SSH jail should use the "ufw" action to integrate with the UFW firewall for consistent rule management.

```
# /etc/fail2ban/jail.local
[DEFAULT]
bantime = 3600
findtime = 600
maxretry = 3
banaction = ufw
ignoreip = 127.0.0.1/8 YOUR_OFFICE_IP

[sshd]
enabled = true
port = 48292
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 7200
```

## 4.2 Custom WebSocket Auth Jail

Create a custom Fail2ban filter to monitor the Ai-Arena Nginx access logs for repeated failed authentication attempts on the WebSocket endpoint. This filter detects patterns such as multiple HTTP 401 responses from the same IP within a short time window, which indicates credential-stuffling attacks against the player authentication system. The filter uses a custom regular expression pattern to match failed WebSocket handshake or authentication failures in the Nginx log format. Configure a more

lenient retry limit (5 attempts within 15 minutes) since legitimate players may occasionally mistype credentials, but set a longer ban time (4 hours) to effectively deter automated attacks.

```
# /etc/fail2ban/filter.d/arena-ws-auth.conf
[Definition]
failregex = ^ .* "GET /ws/arena .*" 401
^ .* "POST /api/auth/login .*" 401
ignoreregex =

# /etc/fail2ban/jail.local (add section)
[arena-ws-auth]
enabled = true
port = http,https
filter = arena-ws-auth
logpath = /var/log/nginx/access.log
maxretry = 5
findtime = 900
bantime = 14400
```

### 4.3 Recidive Jail

The recidive jail is a built-in Fail2ban feature that tracks IP addresses that have been banned multiple times across any jail and imposes escalating penalties. Configure the recidive jail with a very long bantime (1 week for the first recidive offense, permanently for the third) to effectively eliminate persistent attackers. Set the findtime to 86400 seconds (24 hours) and maxretry to 3, meaning an IP that gets banned 3 times in a single day across any service will be placed in the recidive jail. This is particularly effective against botnets that rotate through their IP pool and eventually retry previously banned addresses.

```
[recidive]
enabled = true
banaction = ufw
logpath = /var/log/fail2ban.log
bantime = 604800
findtime = 86400
maxretry = 3
```

## 5. Nginx Reverse Proxy Security

Nginx serves as the reverse proxy and TLS termination point for the Ai-Arena platform, handling all inbound HTTP and WebSocket traffic. As the only publicly accessible service (besides SSH), Nginx is the primary gateway through which all external traffic flows, making its security configuration critically important. A properly hardened Nginx configuration protects against a wide range of web-layer attacks including SQL injection, cross-site scripting (XSS), directory traversal, request smuggling, and denial-of-service conditions caused by oversized requests or excessive connection rates.

### 5.1 Rate Limiting

Implement multi-layer rate limiting to protect against abuse. First, define a rate-limiting zone at the Nginx configuration level that tracks requests per IP address. For the Ai-Arena API endpoints, allow a maximum of 10 requests per second per IP with a burst allowance of 20 requests. For WebSocket connections, implement a separate connection-level limit using the "limit_conn" directive to restrict each IP to a maximum of 5 simultaneous connections. Return HTTP 429 (Too Many Requests) when limits are exceeded, with a JSON error body that matches the API response format so clients can handle it gracefully.

```
# /etc/nginx/nginx.conf (http block)
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_conn_zone $binary_remote_addr zone=ws:10m;

# In server block for /api/ locations:
limit_req zone=api burst=20 nodelay;
limit_conn ws 5;
```

## 5.2 Security Headers

Configure comprehensive HTTP security headers to protect clients against common web vulnerabilities. The Content-Security-Policy (CSP) header restricts which sources of content the browser is allowed to load, preventing XSS attacks by blocking inline scripts and unauthorized resource loading. Set strict-transport-security (HSTS) to enforce HTTPS for all future connections. Disable MIME-type sniffing with X-Content-Type-Options, and prevent clickjacking with X-Frame-Options. The Referrer-Policy header should be set to "strict-origin-when-cross-origin" to avoid leaking sensitive URL parameters in referrer headers.

```
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self';
style-src 'self' 'unsafe-inline';" always;
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains;
preload" always;
```

## 5.3 TLS and WebSocket Proxy Rules

Configure Nginx as a TLS termination proxy for all inbound traffic. The WebSocket proxy configuration must include the "Upgrade" and "Connection" headers to properly handle the HTTP-to-WebSocket protocol upgrade. Set appropriate proxy timeouts for WebSocket connections (read and send timeouts of 3600 seconds for long-lived game connections). Forward the client's real IP address using the X-Real-IP and X-Forwarded-For headers so that the backend application and Fail2ban can correctly identify the originating IP for rate limiting and access control.

```
location /ws/ {
proxy_pass http://127.0.0.1:3000;
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
```

```
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header Host $host;
proxy_read_timeout 3600s;
proxy_send_timeout 3600s;
}
```

### 5.4 Hiding Server Version and Blocking Probes

Remove all identifying information from Nginx response headers to prevent information leakage that attackers use to target known vulnerabilities. Disable the "Server" header entirely by compiling Nginx with the --without-http_server_tokens option, or at minimum set "server_tokens off" in the configuration. Block access to sensitive paths such as /.well-known/security.txt, /server-status, /.git, and common vulnerability scanner paths. Return 444 (Nginx connection close without response) for blocked requests to minimize information leakage. Configure custom error pages that do not reveal server version or framework information.

## 6. SSL/TLS Configuration

Transport Layer Security (TLS) is the cryptographic protocol that protects all communication between players and the Ai-Arena platform. A properly configured TLS stack ensures data confidentiality, integrity, and authenticity for every connection. Misconfigured TLS can undermine the entire security architecture by exposing sensitive data, allowing man-in-the-middle attacks, or supporting legacy protocols with known vulnerabilities. The Ai-Arena platform mandates TLS 1.2 or higher for all connections and uses Let's Encrypt for automated, free certificate management.

### 6.1 Let's Encrypt Setup

Install Certbot and the Nginx plugin to automate certificate issuance and renewal from Let's Encrypt. Certbot handles the ACME challenge-response protocol, proving domain ownership to the certificate authority, and automatically configures Nginx with the obtained certificates. The initial certificate issuance requires that port 80 is accessible and that DNS records for the Ai-Arena domain correctly point to the VPS IP address. Use the "certonly --nginx" mode for maximum control over the Nginx configuration, or "install --nginx" for automatic configuration.

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d arena.yourdomain.com

# Test automatic renewal
sudo certbot renew --dry-run
```

### 6.2 TLS 1.2+ Only and Strong Cipher Suites

Configure Nginx to accept only TLS 1.2 and TLS 1.3 connections, explicitly disabling TLS 1.0 and TLS 1.1 which have known vulnerabilities including BEAST, POODLE, and RC4 biases. Select cipher suites

that provide forward secrecy (ECDHE key exchange), authenticated encryption (GCM or ChaCha20-Poly1305), and strong key lengths (256-bit or higher). The recommended cipher string prioritizes TLS 1.3 cipher suites followed by TLS 1.2 suites with ECDHE-RSA-AES256-GCM-SHA384 as the primary TLS 1.2 choice. Regularly review cipher suite selections against current cryptographic guidance from standards bodies.

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES256-GCM-SHA384:
ECDHE-RSA-AES256-GCM-SHA384:
ECDHE-ECDSA-CHACHA20-POLY1305:
ECDHE-RSA-CHACHA20-POLY1305;
ssl_prefer_server_ciphers off;
ssl_ecdh_curve X25519:P-256:P-384;
```

## 6.3 HSTS and OCSP Stapling

HTTP Strict Transport Security (HSTS) tells browsers to always use HTTPS for the Ai-Arena domain, preventing SSL stripping attacks where an attacker forces a downgrade to HTTP. Set a long max-age (2 years), include subdomains, and submit the domain to the HSTS preload list for maximum protection. OCSP Stapling allows Nginx to include the certificate revocation status in the TLS handshake, eliminating the need for clients to make separate OCSP requests to the certificate authority. This improves both privacy (the CA does not see which clients are connecting) and performance (no additional round-trip for revocation checking). Configure the OCSP staple cache with a shared zone and verify the stapling status using online SSL testing tools.

```
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains;
preload" always;

ssl_stapling on;
ssl_stapling_verify on;
ssl_trusted_certificate /etc/letsencrypt/live/arena.yourdomain.com/chain.pem;
resolver 8.8.8.8 8.8.4.4 valid=300s;
resolver_timeout 5s;
```

## 6.4 Certificate Auto-Renewal

Let's Encrypt certificates are valid for 90 days and must be renewed before expiration. Certbot installs a systemd timer or cron job that automatically attempts renewal twice daily, but only actually renews certificates when they are within 30 days of expiration. Verify the timer is active by checking "systemctl list-timers" and manually run "certbot renew --dry-run" monthly to ensure the renewal process works correctly. Configure a post-renewal hook to reload Nginx after successful certificate renewal: "echo 'systemctl reload nginx' | sudo tee /etc/letsencrypt/renewal-hooks/post/reload-nginx.sh && sudo chmod +x /etc/letsencrypt/renewal-hooks/post/reload-nginx.sh".

# 7. Kernel and Network Hardening

The Linux kernel provides extensive network and memory security controls through the sysctl interface. Properly configuring these kernel parameters significantly reduces the server's vulnerability to network-based attacks including SYN floods, IP spoofing, ICMP redirect manipulation, and various forms of packet injection. These settings complement the application-level and firewall-level protections described in previous sections, providing the deepest layer of defense at the operating system level.

## 7.1 sysctl Parameters

The following sysctl parameters should be configured in /etc/sysctl.d/99-arena-hardening.conf. Each parameter addresses a specific attack vector. Enable SYN cookies to protect against TCP SYN flood denial-of-service attacks without requiring large connection tracking tables. Disable IP forwarding unless the VPS acts as a router or VPN gateway. Disable ICMP redirects to prevent man-in-the-middle attacks via malicious ICMP redirect messages. Enable reverse path filtering to drop packets with spoofed source addresses. Adjust connection tracking parameters to handle the expected connection volume while preventing resource exhaustion.

| Parameter | Value | Purpose |
|---|---|---|
| net.ipv4.tcp_syncookies | 1 | Protect against SYN flood attacks |
| net.ipv4.ip_forward | 0 | Disable routing (VPS is endpoint only) |
| net.ipv4.conf.all.accept_redirects | 0 | Block ICMP redirect attacks |
| net.ipv4.conf.all.rp_filter | 1 | Reverse path filtering (anti-spoofing) |
| net.ipv4.conf.all.log_martians | 1 | Log spoofed packets for monitoring |
| net.netfilter.nf_conntrack_max | 65536 | Connection tracking table size |
| net.ipv4.tcp_max_syn_backlog | 4096 | SYN queue size for burst handling |
| net.ipv4.tcp_fin_timeout | 15 | Reduce FIN_WAIT timeout (faster reclaim) |
| net.ipv4.tcp_keepalive_time | 300 | TCP keepalive interval (5 minutes) |
| net.core.somaxconn | 1024 | Maximum pending connection queue |
| kernel.dmesg_restrict | 1 | Restrict dmesg to root only |
| kernel.kptr_restrict | 2 | Hide kernel pointer addresses |

```
# Apply all sysctl settings
sudo sysctl --system
```

## 7.2 IPv6 Considerations

If the VPS has IPv6 connectivity, apply equivalent security rules for IPv6 traffic. The UFW firewall supports IPv6 rules alongside IPv4 rules, and the same default-deny policy should be applied to IPv6. Many automated attack tools still focus on IPv4, but IPv6-based attacks are becoming increasingly common. Ensure that only necessary IPv6 services are exposed and that IPv6 connection tracking is enabled. If IPv6 is not needed, disable it entirely by adding "ipv6.disable=1" to the kernel boot parameters or by blacklisting the ipv6 kernel module. This reduces the attack surface and eliminates potential misconfigurations in IPv6 firewall rules.

### 7.3 TCP Hardening

Additional TCP-level hardening includes disabling source routing (net.ipv4.conf.all.accept_source_route = 0) to prevent attackers from specifying the return path for packets, and setting net.ipv4.conf.all.send_redirects = 0 to prevent the server from sending ICMP redirect messages. Enable TCP timestamps (net.ipv4.tcp_timestamps = 1) for protection against wrapped sequence number attacks, but disable TCP window scaling on low-memory VPS instances if connection tracking memory is a concern. Review and tune these settings periodically based on traffic patterns and any changes to the threat landscape. Use "sysctl -a | grep net.ipv4" to audit all current TCP settings.

## 8. Application Security

Application security encompasses the protection of sensitive configuration values, runtime secrets, file permissions, database access controls, and session management within the Ai-Arena platform itself. Even with a perfectly hardened network perimeter, a single application-level vulnerability can lead to full compromise. This section addresses the critical application-layer security measures that protect the Ai-Arena platform code, data, and user sessions from exploitation.

### 8.1 ARENA_ENC_KEY Management

The ARENA_ENC_KEY is the master encryption key used by the Ai-Arena platform to encrypt all game state data, player communications, and WebSocket traffic. This key must be treated with the same rigor as a root password. Generate the key using a cryptographically secure random number generator with at least 256 bits of entropy. Store the key in a dedicated file outside the application codebase with strict file permissions (readable only by the application service user). Never commit the encryption key to version control, include it in configuration files that are synced to backup systems, or transmit it over unencrypted channels. Implement key rotation procedures (see Section 9) and maintain an offline backup of the key in a physically secure location such as a hardware security module (HSM) or a sealed envelope in a safe.

```
# Generate 256-bit encryption key
openssl rand -hex 32 > /opt/arena/.arena_enc_key
chmod 600 /opt/arena/.arena_enc_key
chown arena-app:arena-app /opt/arena/.arena_enc_key
```

## 8.2 Environment Variable Protection

All sensitive configuration values, including database credentials, API keys, Redis passwords, and the encryption key path, must be passed to the application through environment variables defined in a secure .env file. The .env file must have 600 permissions and be owned by the application service user only. Add the .env file to .gitignore to prevent accidental commits. Use the "dotenv" library or equivalent to load environment variables at application startup. Never hardcode secrets in source code, and never log environment variable values. Implement a startup check that verifies all required environment variables are defined and that the encryption key file exists and is readable before the application begins accepting connections.

## 8.3 File Permissions

Apply the principle of least privilege to all files on the VPS. The application source code directory should be owned by the arena-app user with 750 permissions (rwxr-x---). Configuration files containing secrets should use 600 permissions (rw-------). Log files should use 644 permissions owned by the application user. The /opt/arena directory tree should not be readable by other users on the system. Use "find /opt/arena -type f -perm /o=r" to audit for files that are world-readable. Regularly review file permissions with automated tools and integrate permission checks into the deployment pipeline to catch configuration drift before it reaches production.

## 8.4 Database Security

The PostgreSQL database must be configured to listen only on localhost (127.0.0.1) and never on the public network interface. Create a dedicated database user for the Ai-Arena application with limited privileges: SELECT, INSERT, UPDATE, DELETE on the arena schema only. The database superuser (postgres) should have a strong password and be used only for administrative tasks. Enable SSL for database connections even on localhost to protect credentials in transit. Configure connection pooling with PgBouncer to limit the maximum number of simultaneous database connections and prevent connection exhaustion attacks. Enable PostgreSQL audit logging to record all DDL changes and failed authentication attempts for forensic analysis.

## 8.5 Session Management

The Ai-Arena platform uses Redis-backed session storage with cryptographically signed session tokens. Sessions must have a maximum lifetime of 24 hours for player sessions and 8 hours for administrative sessions, with automatic expiration enforced by Redis TTL. Implement session fixation protection by regenerating session IDs after successful login. Store only the session ID in the client cookie (not session data) and set the Secure, HttpOnly, and SameSite=Strict flags on all session cookies. Implement session invalidation on password change and logout, and maintain a revocation list for compromised sessions that must be immediately terminated.

# 9. Encryption Architecture

The Ai-Arena platform employs a layered encryption architecture designed to protect all data at rest and in transit. The encryption system is built on industry-standard algorithms and protocols, with additional proprietary measures for traffic analysis resistance and anti-replay protection. Understanding the encryption architecture is essential for maintaining system integrity and responding to security incidents that may involve cryptographic material.

## 9.1 AES-256-GCM Encryption

All game state data, player messages, and WebSocket payloads are encrypted using AES-256-GCM (Galois/Counter Mode). AES-256 provides 256-bit key strength, which is considered computationally infeasible to brute-force with current and foreseeable technology. GCM mode provides both confidentiality and integrity in a single operation, generating a 128-bit authentication tag that verifies the data has not been tampered with during transmission. The combination of AES-256 key size and GCM authentication tag provides 256 bits of confidentiality and 128 bits of integrity protection. The ARENA_ENC_KEY is split into two components: a 256-bit encryption key and a derived 128-bit authentication key using HKDF (HMAC-based Key Derivation Function) with SHA-256.

## 9.2 Anti-Replay Nonce Tracking

Every encrypted message includes a unique 96-bit nonce (number used once) that serves as the initialization vector for AES-GCM. The nonce must never be reused with the same key, as this would catastrophically compromise the encryption. The Ai-Arena platform maintains an in-memory nonce tracking table (backed by Redis for persistence across server restarts) that records all recently used nonces. When a message is received, the nonce is checked against this table; if it has been seen before, the message is rejected as a potential replay attack. Nonces older than the maximum message lifetime (typically 60 seconds for real-time game data) are automatically pruned to prevent unbounded memory growth. This mechanism ensures that captured and retransmitted packets cannot be replayed to manipulate game state.

## 9.3 Traffic Padding

To resist traffic analysis attacks that could reveal information about game state based on packet sizes and timing, the Ai-Arena platform implements traffic padding. Each encrypted WebSocket message is padded to a standard block size (multiples of 256 bytes) before encryption. This means that a 50-byte "move" command and a 200-byte "state update" command both appear as 256-byte encrypted payloads, making it impossible for an observer to distinguish between message types based on size. Additionally, noise packets (random encrypted data) are periodically injected into the stream at random intervals to obscure the actual communication pattern and prevent timing analysis that could correlate specific packet bursts with game events.

## 9.4 Noise Packet Injection

Noise packets are randomly generated encrypted messages that contain no game data but are indistinguishable from legitimate encrypted messages to any observer without the encryption key. The Ai-Arena platform injects noise packets at a rate of approximately one every 5 to 15 seconds per active WebSocket connection, with the exact interval determined by a cryptographically secure random number generator. This rate is calibrated to add sufficient entropy to the traffic pattern without significantly impacting bandwidth or latency. Noise packets use the same AES-256-GCM encryption and nonce tracking system as legitimate messages, ensuring they are computationally indistinguishable from real data even under sophisticated cryptanalysis.

### 9.5 Key Rotation Procedures

Encryption keys must be rotated on a regular schedule to limit the impact of potential key compromise. The Ai-Arena platform supports online key rotation without service interruption using a dual-key system: the current key and the previous key are both maintained in memory. During rotation, new messages are encrypted with the new key while incoming messages encrypted with the previous key are still accepted for a transition period of 24 hours. The rotation procedure involves generating a new key, updating the application configuration, and gracefully reloading the application process. The old key is securely overwritten in memory after the transition period expires. Key rotation should be performed at least quarterly, and immediately if there is any suspicion of key compromise.

## 10. Monitoring and Audit

Continuous monitoring and regular auditing are essential components of a comprehensive security posture. Detection is the complement to prevention: while hardening measures reduce the attack surface, monitoring ensures that any breach or attempted breach is identified quickly enough to respond effectively. The Ai-Arena platform implements a multi-layered monitoring strategy covering system logs, file integrity, process health, and network activity. Alerts are configured to notify administrators of suspicious activity in near-real-time, enabling rapid incident response before attackers can establish persistence or exfiltrate data.

### 10.1 Log Files to Monitor

The following log files should be continuously monitored for security events. Authentication logs (/var/log/auth.log) record all SSH login attempts and sudo command executions, and are the primary source for detecting brute-force attacks and unauthorized access. Nginx access and error logs (/var/log/nginx/access.log and /var/log/nginx/error.log) record all HTTP requests and server errors, revealing scanning activity, SQL injection attempts, and abnormal request patterns. Application logs (/opt/arena/logs/) record game events, authentication failures, and internal errors. System logs (/var/log/syslog) record kernel messages, service start/stops, and hardware events. Fail2ban logs (/var/log/fail2ban.log) record all ban and unban events. PostgreSQL logs (/var/log/postgresql/) record database queries, connection events, and errors.

| Log File | Key Events to Watch |
|---|---|
| /var/log/auth.log | Failed SSH logins, sudo usage, new user creation |
| /var/log/nginx/access.log | HTTP 4xx/5xx spikes, unusual User-Agents, path scans |
| /var/log/nginx/error.log | Upstream timeouts, SSL handshake failures |
| /var/log/fail2ban.log | Ban events, jail failures, ignoreip matches |
| /opt/arena/logs/app.log | Authentication failures, encryption errors, anomalies |
| /var/log/postgresql/main.log | Failed connections, DDL changes, slow queries |
| /var/log/syslog | OOM kills, service crashes, kernel warnings |

## 10.2 Logwatch Configuration

Install and configure Logwatch to generate daily security summary reports that are emailed to the security team. Logwatch aggregates and filters log entries, producing a concise human-readable report that highlights the most important events from the previous 24 hours. Configure Logwatch with the "high" detail level for SSH and Nginx services, and "medium" for other services. The daily report should include: total SSH login attempts (successful and failed), top attacking IPs, Fail2ban ban summary, Nginx error summary, and any kernel warnings. Configure Logwatch to ignore routine events and focus on anomalies. Integrate with a mail relay or alerting service to ensure reports are delivered reliably even if the VPS mail system is blocked.

## 10.3 Intrusion Detection with AIDE

Advanced Intrusion Detection Environment (AIDE) is a file integrity checker that monitors critical system and application files for unauthorized modifications. AIDE creates a cryptographic database of file hashes, permissions, and metadata for all monitored files, then periodically compares the current state against this baseline. Any discrepancy indicates potential tampering by an attacker or unauthorized configuration change. Configure AIDE to monitor /etc/, /opt/arena/, /usr/bin/, /usr/sbin/, and all binary directories. Initialize the database after a clean installation, and store the baseline database on read-only media or a separate system. Run AIDE checks daily via cron and send the results to the security team. Investigate any reported changes immediately, as they may indicate that an attacker has modified system binaries or configuration files to establish persistence.

```
sudo apt install aide -y
sudo aideinit
sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db

# Daily check via cron (added to /etc/cron.daily/aide-check)
#!/bin/bash
/usr/bin/aide --check | mail -s "AIDE Report $(date)" security@arena.com
```

## 10.4 Monitoring with PM2

The Ai-Arena application process is managed by PM2, which provides process monitoring, automatic restart on crash, log rotation, and CPU/memory monitoring. Configure PM2 to restart the application automatically on crash (the default behavior) with a maximum restart limit of 10 times within a 60-second window to prevent restart loops. Enable PM2 log rotation to prevent log files from consuming excessive disk space. Configure PM2 to send alerts on process exit, high CPU usage (>90% sustained), and high memory usage. Use "pm2 startup" to register PM2 as a system service so it starts automatically on boot and the application survives server restarts.

```
pm2 start /opt/arena/server.js --name ai-arena --max-memory-restart 512M
pm2 save
pm2 startup
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
```

# 11. Incident Response

Despite the best preventive measures, security incidents can and do occur. A well-defined incident response plan ensures that the security team can respond quickly and effectively to minimize damage, preserve evidence, and restore normal operations. The Ai-Arena incident response plan defines clear procedures for identification, containment, eradication, recovery, and post-incident analysis. All team members with administrative access should be familiar with these procedures and participate in regular incident response drills.

## 11.1 Signs of Compromise

The following indicators may suggest a security compromise and should trigger the incident response procedure: unexpected user accounts in /etc/passwd, unauthorized SSH authorized_keys entries, modified system binaries (detected by AIDE), unexpected listening ports (detected by "ss -tulnp" or "netstat -tulnp"), unusual network connections to unknown IP addresses, sudden spikes in CPU or memory usage with no corresponding application load, modified crontab entries, unexpected sudo log entries from unknown users, database records with anomalous values, and player reports of unusual game behavior that suggests manipulation. Any combination of these indicators should be treated as a confirmed compromise until proven otherwise.

## 11.2 Backup Procedures

Maintain regular backups of all critical data and configuration files using a 3-2-1 backup strategy: three copies of data, on two different media types, with one copy offsite. Daily incremental backups should capture database dumps, application logs, and configuration files. Weekly full backups should capture the entire /opt/arena directory and /etc directory. Backups must be encrypted before transfer using AES-256 encryption, and the backup encryption key must be stored separately from the backups

themselves. Test backup restoration procedures quarterly to ensure that backups are complete and restorable. Use a tool like Duplicity or BorgBackup for encrypted, deduplicated, incremental backups to a remote storage location (S3, Backblaze B2, or a dedicated backup server).

### 11.3 Emergency Lockdown Steps

If a compromise is suspected or confirmed, execute the following lockdown procedure immediately. First, block all inbound traffic except from your trusted administration IP using emergency UFW rules. This stops the attacker from exfiltrating data or establishing additional backdoors. Second, disable all non-essential services: stop the application, database, and Redis services. Third, change all passwords and rotate the ARENA_ENC_KEY. Fourth, take a forensic snapshot of the system before making any changes (create a full disk image or at minimum capture running processes, network connections, and file system metadata). Fifth, review all user accounts, SSH keys, and cron jobs for unauthorized additions. Document every action taken during the incident for the post-incident report.

```
# Emergency lockdown commands
sudo ufw default deny incoming
sudo ufw allow from YOUR_ADMIN_IP to any port 48292
sudo systemctl stop ai-arena
sudo systemctl stop postgresql
sudo systemctl stop redis-server

# Capture forensic data
sudo ss -tulnp > /tmp/forensic_connections.txt
ps auxf > /tmp/forensic_processes.txt
sudo last -50 > /tmp/forensic_logins.txt
```

### 11.4 Forensic Preservation

Before attempting to clean or restore a compromised system, preserve all available forensic evidence. Create a binary image of the entire disk using "dd" or a forensic tool like FTK Imager to an external storage device. Capture volatile data first (running processes, network connections, memory dump) as this data is lost on reboot. Preserve all log files, including rotated and compressed logs, by copying them to read-only storage. Do not modify any files on the compromised system until forensic imaging is complete. If a full disk image is not feasible due to storage constraints, at minimum preserve /var/log/, /etc/, /tmp/, /opt/arena/logs/, and the output of system information commands. All preserved evidence should be hashed using SHA-256 to establish a chain of custody and verify integrity during analysis.

## 12. Maintenance Schedule

Security is not a one-time configuration but a continuous process that requires regular maintenance, updates, and reviews. The Ai-Arena platform maintenance schedule defines daily, weekly, monthly, and quarterly tasks that ensure the security configuration remains current, vulnerabilities are patched promptly, and operational hygiene is maintained. Automated tools handle routine tasks, but human review is essential for detecting subtle issues that automation might miss. All maintenance activities

should be logged and reviewed during quarterly security audits.

## 12.1 Daily Tasks

Daily security tasks are primarily automated but should be reviewed by an administrator. Review Logwatch email reports for unusual authentication attempts, service errors, and system warnings. Verify that Fail2ban is running and that the ban list contains only expected entries. Check PM2 process status to ensure the application is running normally. Monitor disk space usage with "df -h" to prevent log files or backups from filling the disk. Review the first and last entries in /var/log/auth.log for unauthorized access attempts. Verify that the TLS certificate is still valid (check expiration date with "openssl s_client -connect arena.yourdomain.com:443 2>/dev/null | openssl x509 -noout -dates"). These checks take approximately 5-10 minutes and should be part of every administrator's daily routine.

## 12.2 Weekly Tasks

Weekly tasks include a more thorough review of system security. Review Fail2ban banned IP lists and investigate repeat offenders who may be targeting the platform specifically. Check AIDE file integrity reports for any unauthorized modifications. Review Nginx access logs for scanning patterns or unusual request paths. Check system updates with "apt list --upgradable" and apply security updates if they address critical vulnerabilities. Review database query logs for slow queries or unusual access patterns. Verify that automated backups completed successfully by checking the backup job logs and testing restoration of a single file. Review user accounts and SSH authorized_keys for any unexpected additions. These weekly reviews should take approximately 30-45 minutes and should be documented in a shared security log for audit purposes.

## 12.3 Monthly Tasks

Monthly maintenance includes patching and deeper security reviews. Apply all outstanding security updates with "sudo apt update && sudo apt upgrade -y" and reboot the server if kernel updates were applied. Review and rotate the ARENA_ENC_KEY following the rotation procedure in Section 9.5. Test the backup restoration procedure by restoring the most recent backup to a test environment and verifying application functionality. Review Fail2ban configuration and adjust thresholds based on the volume of false positives observed over the past month. Audit all user accounts, sudo privileges, and API keys, and revoke any that are no longer needed. Review SSL/TLS configuration using an external scanner such as SSL Labs SSL Test and address any reported issues. Generate and review a comprehensive security metrics report covering incidents, response times, and trend analysis.

## 12.4 Quarterly Tasks

Quarterly tasks involve strategic security reviews and updates. Conduct a full security audit of the VPS configuration, comparing the current state against the baseline established in this guide. Review and update the incident response plan based on lessons learned from any incidents during the quarter. Perform a penetration test or vulnerability assessment using automated tools such as Lynis, OpenVAS,

or Nikto. Review the firewall rules and remove any rules that are no longer necessary. Audit the encryption configuration and verify that all cryptographic parameters meet current best practices. Update the documentation to reflect any configuration changes made during the quarter. Conduct an incident response tabletop exercise with the team to practice the procedures defined in Section 11. Review the maintenance schedule itself and adjust task frequency based on operational experience and evolving threat intelligence.

| Frequency | Task | Estimated Time |
|---|---|---|
| Daily | Review Logwatch reports, check Fail2ban/PM2 status, verify TLS cert | 5-10 min |
| Weekly | Review AIDE reports, Nginx logs, apply critical patches | 30-45 min |
| Monthly | Apply all security updates, rotate encryption key, test backups | 2-3 hours |
| Quarterly | Full security audit, penetration test, incident response drill | 1-2 days |

Reminder: This guide represents the security baseline at the time of writing. Security is an evolving discipline, and new threats, vulnerabilities, and best practices emerge continuously. Schedule regular reviews of this guide and update it as needed to reflect changes in the threat landscape and platform architecture.